Presentation Transcript (5 minutes)

Review 3 deck · Adaptive CPU Scheduling Using Predicted Burst Lengths
Shivaansh Aggarwal (24BCE0883) · Devansh Sharma (24BCE0717)

Shivaansh: slides 1, 2 (first half), 3, 5, 7, 9, 10. **Devansh:** slides 2 (second half), 4, 6, 8. *Italics = what to do, not what to say.* The slide numbers match **Review_3.pptx**.

1 – Title 15 s

Shivaansh

Good morning ma'am. I'm Shivaansh, and this is Devansh. Our project is a CPU scheduler that **predicts how long a program will run next**, and uses that to decide **which program runs and for how long**.

2 – Recap: problem and approach 45 s

Shivaansh

The classical algorithms are **static**. FCFS gives **17 ms** average wait on the 24, 3, 3 example; SJF gives **3 ms** but needs the next burst, which **the OS doesn't know**; and Round Robin's quantum is **fixed by hand**. The usual estimate, exponential averaging, **lags** when a process changes behaviour.

Devansh

So we **predict the next burst with a gradient boosting model**, which is **14% more accurate**, and use it twice: to pick **who runs**, with **ageing** so nothing starves, and to set **how long it runs** using the textbook **80% rule, automatically**.

3 – Experimental setup 25 s

Shivaansh

We tested **6 workload types, 11 schedulers and 10 seeds, 660 paired runs**, so every scheduler sees exactly the same processes. The model and the ageing weight were tuned on **separate seeds**, never on the test data.

4 – The simulator matches the textbook 25 s

Devansh

Before any ML, we verified the simulator: FCFS gives exactly **17**, shortest-first **3**, Round Robin **5.67** and SRTF **6.5**, and **all 37 tests pass**.

5 – Main result 30 s

Shivaansh

On the mixed workload, the realistic case, our scheduler has **27% less waiting than SJF**, and it is better on **all 10 seeds**. It closes **54% of the gap to the oracle**, which knows the future, and uses **61% fewer context switches** than Round Robin with 5 ms.

6 – Live run 30 s

Devansh

Optional: run `python demo.py --part 3` in Terminal; otherwise point at the screenshot.

This is a real run of all **11 schedulers on one workload**. Ours, highlighted, waits about **1,900 ms against 2,800 for SJF**, with a lower worst case and **far fewer switches** than Round Robin.

7 – Results across workloads 30 s

Shivaansh

Across all six workloads, the clear win is the **mixed workload**. **Ageing halves the worst-case wait** on CPU-bound and phase-changing work. To be honest about the limits, it is **14% worse on all-I/O work**, where bursts are tiny and random.

8 – Why it works 30 s

Devansh

The quantum **follows the workload**: about **6 ms for interactive, close to 100 ms for CPU-bound and 3.5 ms for I/O-bound**. Switching parts off shows that **without ageing the worst case doubles**, and that prediction alone gives only 5%. **The gain comes from using the prediction twice.**

9 – Comparison with existing schedulers 30 s

Shivaansh

Against existing schedulers, ours has the **lowest average wait except SRTF**, which is 3% better on average but has a **35% higher worst case**. MLFQ is close but needs **twice the switches**. And unlike the papers we studied, we **never assume burst lengths are known**.

10 – Conclusion and future work 25 s

Shivaansh

So **applying the textbook quantum rule automatically from predictions works**: **27% less waiting** on mixed workloads, with the worst case halved. Next, we'd test on **real program traces** and move it into **xv6**. **Thank you, ma'am.**

Total: about 4.5 minutes.

If asked – one-line answers

Why ML? (Devansh) **12–22% more accurate** than exponential averaging, but the big gain comes from **using the prediction twice**.

Is the simulator correct? (Shivaansh) **Exact textbook answers** and **37 tests pass**.

What if the prediction is wrong? (Devansh) The process still runs; **ageing guarantees it gets the CPU**, and the quantum corrects with **recent errors**.

Did you tune on test data? (Devansh) No, training and tuning used **separate seeds**.

Real OS? (Shivaansh) It's a **controlled experiment**; a kernel would need a smaller model, and **xv6** is next.